

# Lab Assignment -20

**Name:** CH VISHWANATH RAO

**HT NO:** 2303A51095

**Batch No. :** 02

## Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

*Lab Objectives:*

- *Understand how to test AI-generated code for common security vulnerabilities.*
- *Learn to apply secure coding principles while analysing AI outputs.*
- *Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.*
- *Enhance code reliability and safety by using AI for secure refactoring.*

### Task 1 – Password Storage Vulnerability Check

*Task: Analyse an AI-generated signup script for insecure password storage.*

**Instructions:**

- *Prompt AI to generate a Python user registration program.*
- *Check whether passwords are stored in plain text.*
- *Improve the script using hashing (bcrypt or hashlib).*

**Expected Output:**

- *A secure registration script with hashed password storage.*

```

#generate a python user registration program . check whether passwords are stored in plain text. Improve the script using hashing (bcrypt or hashlib).
import bcrypt
def create_account(user_name, user_password):
    # Hash the password using bcrypt
    hashed_pass = bcrypt.hashpw(user_password.encode('utf-8'), bcrypt.gensalt())

    # Store the username and hashed password in a dictionary (simulating a database)
    account_data = {
        'user_name': user_name,
        'user_password': hashed_pass
    }

    return account_data
# Example usage
user_name = input("Enter your user name: ")
user_password = input("Enter your password: ")
account_info = create_account(user_name, user_password)
print("Account created successfully!")
print("Stored account data:", account_info)

```

**Output :**

```

Enter your username: Vishwarao
Enter your password: hibro@123
User registered successfully!
Stored user data: {'username': 'Vishwarao', 'password': b'$2b$12$uKMHaFIL7SUhwaA2RAkkzOHQDKj74EeiUmBmr4rC7QbwuNx5jGo9e'}

```

## Task 2 – Hardcoded Secrets Detection

*Task: Evaluate AI-generated code for hardcoded API keys or credentials.*

### Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

### Expected Output:

- A secure script with secrets managed properly.

```
#generate a script that connects to an external API. Identify if API keys/passwords are written directly in the code. If so, refactor the code to use environment variable
import os
import requests
def get_api_data():
    # Load API key from environment variable
    api_key = os.getenv('API_KEY')
    if not api_key:
        raise ValueError("API_KEY environment variable not set")
    # Make a request to the external API using the API key
    url = "https://api.example.com/data"
    headers = {
        "Authorization": f"Bearer {api_key}"
    }
    response = requests.get(url, headers=headers)
    return response.json()
# Example usage
if __name__ == "__main__":
    try:
        data = get_api_data()
        print("API data retrieved successfully!")
        print(data)
    except Exception as e:
        print(f"Error: {e}")
```

**Output :**

```
PS C:\Users\chvis\OneDrive\Desktop\ai\LAB_20.py> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/LAB_20.py/assig_20.py
Error: API_KEY environment variable not set
```

### Task 3 – File Upload Security Validation

*Task: Test an AI-generated file upload program for unsafe file handling. Instructions:*

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename Handling.

**Expected Output:**

- A secure file upload implementation.

```
from flask import Flask as WebFramework, request as http_request
from werkzeug.utils import secure_filename as sanitize_name
import os as operating_system

web_service = WebFramework(__name__)

FILE_STORAGE_DIRECTORY = 'uploads/'
PERMITTED_FILE_TYPES = ('png', 'jpg', 'jpeg', 'gif')
web_service.config['UPLOAD_FOLDER'] = FILE_STORAGE_DIRECTORY
def is_supported_file(uploaded_name):
    return (
        '.' in uploaded_name
        and uploaded_name.rsplit('.', 1)[1].lower() in PERMITTED_FILE_TYPES
    )
@web_service.route('/upload', methods=['GET', 'POST'])
def process_file_submission():
    if http_request.method == 'POST':
        # Ensure file field exists in request
        if 'file' not in http_request.files:
            return "No file part"

        selected_upload = http_request.files['file']

        # Handle empty filename submission
        if selected_upload.filename == '':
            return "No selected File"

        if selected_upload and is_supported_file(selected_upload.filename):
            safe_upload_name = sanitize_name(selected_upload.filename)
            destination_path = operating_system.path.join(
                web_service.config['UPLOAD_FOLDER'],
                safe_upload_name
            )
            selected_upload.save(destination_path)
            return "File uploaded successfully!"
        return "Invalid file type. Allowed types are: png, jpg, jpeg, gif."

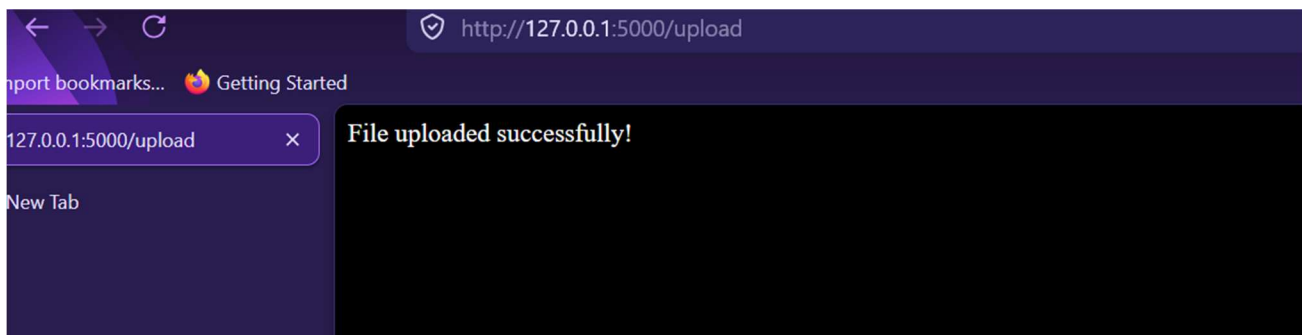
    return '''
<doctype html>
<title>Upload a File</title>
<h1>Upload a File</h1>
<form method=post enctype=multipart/form-data>
  <input type=file name=file>
  <input type=submit value=Upload>
</form>
'''
```

FILES OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\chvis\OneDrive\Desktop\ai\LAB\_20.py> & C:\Python314\python.exe c:\Users\chvis\OneDrive\Desktop\ai\LAB\_20.py\assig\_20.py

Output :

```
PS C:\Users\chvis\OneDrive\Desktop\ai\LAB_20.py> & C:\Python314\python.exe c:\Users\chvis\OneDrive\Desktop\ai\LAB_20.py\assig_20.py
* Serving Flask app 'assig_20'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 818-988-512
```



## Task 4 – Command Injection Prevention

*Task: Analyze an AI-generated Python script that executes system commands.*

**Instructions:**

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.

- Fix using safe subprocess calls and input validation.

### Expected Output:

- A secure command execution script.

### CODE:

```
import subprocess
def run_command(user_input):
    # Validate user input to prevent command injection
    if not user_input.isalnum():
        raise ValueError("Invalid input: only alphanumeric characters are allowed.")

    # Use subprocess to run a safe command
    result = subprocess.run(
        f"echo {user_input}",
        shell=True,
        capture_output=True,
        text=True
    )
    return result.stdout.strip()

# Example usage
if __name__ == "__main__":
    user_input = input("Enter a string to echo: ")
    try:
        output = run_command(user_input)
        print("Command output:", output)
    except ValueError as e:
        print(f"Error: {e}")
```

### OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai\LAB_20.py> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/LAB_20.py/assig_20.py
Enter a string to echo: TAXICAR
Command output: TAXICAR
```

## Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation. Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitisation.

### Expected Output:

A secure form resistant to injection attacks

```

from flask import Flask, request, render_template_string
import html
app = Flask(__name__)
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        # Get user input and sanitize it
        user_feedback = request.form.get('feedback', '')
        sanitized_feedback = html.escape(user_feedback)

        # Here you would typically save the feedback to a database
        return f"Thank you for your feedback: {sanitized_feedback}"

    # Render the feedback form
    return '''
<!doctype html>
<title>Feedback Form</title>
<h1>Feedback Form</h1>
<form method=post>
  <textarea name=feedback rows=4 cols=50></textarea><br>
  <input type=submit value=Submit>
</form>
'''
if __name__ == '__main__':
    app.run(debug=True)

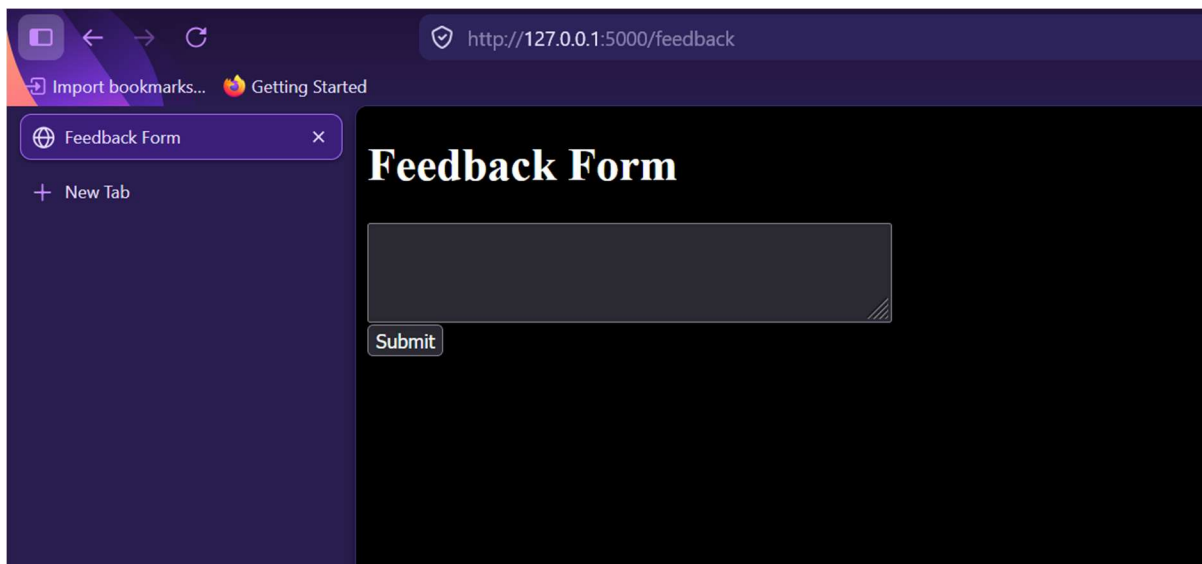
```

OUTPUT:

```

PS C:\Users\chvis\OneDrive\Desktop\ai\LAB_20.py> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/LAB_20.py/assig_20.py
* Serving Flask app 'assig_20'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 818-988-512
127.0.0.1 - - [04/Apr/2026 23:32:56] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 23:32:56] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 23:33:01] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 23:33:02] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 23:33:02] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [04/Apr/2026 23:33:03] "GET / HTTP/1.1" 404 -

```



The screenshot shows a web browser window with the address bar displaying `http://127.0.0.1:5000/feedback`. The browser's interface includes a sidebar with a 'Feedback Form' tab and a 'New Tab' button. The main content area displays the title 'Feedback Form' in a large, bold, serif font. Below the title is a large, empty text input area with a light gray border and a small diagonal line in the bottom right corner. At the bottom of the text area is a 'Submit' button with a light gray background and a dark gray border.

